

GKE 的 Jenkins 多分支管道

來源：<https://codelabs.developers.google.com/codelabs/jenkins-pipeline-gke?hl=zh-tw>

步驟數：9

本程式碼研究室會引導使用者逐步在 GKE 上部署 Jenkins 執行個體，包括自動調整建構代理程式。

目錄

1. [1. 總覽](#)
2. [2. 開始設定](#)
3. [3. 建立 Kubernetes 叢集](#)
4. [4. 使用 Helm 部署 Jenkins](#)
5. [5. 安裝及設定 GKE 外掛程式](#)
6. [6. 設定 Jenkins](#)
7. [7. 設定管道](#)
8. [8. 清除所用資源](#)
9. [9. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽

Jenkins 是最受歡迎的持續整合解決方案之一。這項工具可自動執行軟體開發流程中非人為的重要部分。將 Jenkins 部署至 Google Cloud 的 Kubernetes，並使用 GKE 外掛程式，我們就能在需要時快速自動調度建構執行器。搭配 Cloud Storage，我們就能輕鬆建構及測試應用程式。

執行步驟

- 將 Jenkins 部署至 Kubernetes 叢集
- 部署及設定 Jenkins GKE 外掛程式，讓 Jenkins 能夠建立及終止 Pod 做為執行器節點
- 建構及測試範例 SpringBoot 應用程式
- 建構容器並發布至 Google Container Registry
- 將範例應用程式部署至測試和正式 GKE 環境

軟硬體需求

- 已設定帳單的 Google Cloud 專案。如果沒有，請[建立一個](#)。

注意：本程式碼研究室部署的解決方案會產生計費活動。如要瞭解 Google Compute Engine 定價，請參閱[這篇文章](#)。Google Cloud Platform 新使用者享有價值 \$300 美元的免費試用期。

步驟 2 · 約 3 分鐘

2. 開始設定

本程式碼研究室完全可在 Google Cloud Platform 上執行，不需要在本機安裝或設定任何項目。

Cloud Shell

在本程式碼實驗室中，我們將使用 [Cloud Shell](#)，透過指令列佈建及管理不同的雲端資源和服務。

啟用 API

我們需要在專案中啟用下列 API：

- **Compute Engine API** - 建立及執行虛擬機器
- **Kubernetes Engine API** - 建構及管理容器型應用程式
- **Cloud Build API** - Google Cloud 的持續整合和持續推送軟體更新平台
- **Service Management API**：服務生產者可透過此 API 在 Google Cloud Platform 上發布服務
- **Cloud Resource Manager API** - 建立、讀取及更新 Google Cloud 資源容器的中繼資料

執行下列 gcloud 指令，啟用必要的 API：



```
gcloud services enable compute.googleapis.com \  
container.googleapis.com \  
cloudbuild.googleapis.com \  
servicemanagement.googleapis.com \  
cloudresourcemanager.googleapis.com \  
--project ${GOOGLE_CLOUD_PROJECT}
```

建立 GCS bucket

我們需要 GCS bucket 來上傳測試作業。讓我們在名稱中使用專案 ID 建立 bucket，確保名稱不重複：

```
gsutil mb gs://${GOOGLE_CLOUD_PROJECT}-jenkins-test-bucket/
```

注意：bucket 名稱在全域範圍內都不可重複。如果上述指令傳回 409 服務例外狀況錯誤，請使用其他名稱重試。

步驟 3 · 約 10 分鐘

3. 建立 Kubernetes 叢集

建立叢集

接著，我們要建立 GKE 叢集來代管 Jenkins 系統，包括將做為工作站節點調度的 Pod。 `--scopes` 旗標所指出的額外範圍，可讓 Jenkins 存取 Cloud Source Repositories 和 Container Registry。在 Cloud Shell 中執行下列指令：

```
gcloud container clusters create jenkins-cd \
--machine-type n1-standard-2 --num-nodes 1 \
--zone us-east1-d \
--scopes "https://www.googleapis.com/auth/source.read_write,cloud-platform" \
--cluster-version latest
```

我們也來部署 2 個叢集，用於代管範例應用程式的暫存和正式版建構作業：

```
gcloud container clusters create staging \
--machine-type n1-standard-2 --num-nodes 1 \
--zone us-east1-d \
--cluster-version latest
```

```
gcloud container clusters create prod \
--machine-type n1-standard-2 --num-nodes 2 \
--zone us-east1-d \
--cluster-version latest
```



驗證

叢集建立完成後，我們可以使用 `gcloud container clusters list` 確認叢集正在執行：

輸出內容的 `STATUS` 欄應包含 `RUNNING`：

NAME	LOCATION	MASTER_VERSION	MASTER_IP	MACHINE_TYPE	NODE_VERSION	NUM_NODES	STATUS
jenkins-cd	us-east1-d	1.15.9-gke.9	34.74.77.124	n1-standard-2	1.15.9-gke.9	2	RUNNING
prod	us-east1-d	1.15.9-gke.9	35.229.98.12	n1-standard-2	1.15.9-gke.9	2	RUNNING
staging	us-east1-d	1.15.9-gke.9	34.73.92.228	n1-standard-2	1.15.9-gke.9	2	RUNNING

步驟 4 · 約 5 分鐘

4. 使用 Helm 部署 Jenkins

安裝 Helm

我們會使用 Kubernetes 的應用程式套件管理工具 Helm，在叢集上安裝 Jenkins。如要開始使用，請下載專案，其中包含用於部署 Jenkins 的 Kubernetes 資訊清單：

```
git clone https://github.com/GoogleCloudPlatform/continuous-deployment-on-kubernetes.git ~/continuous-deployment-on-kubernetes
```

將目前的工作目錄變更為專案目錄：

```
cd ~/continuous-deployment-on-kubernetes/
```

建立叢集角色繫結，授予自己叢集管理員角色權限：

```
kubectl create clusterrolebinding cluster-admin-binding --clusterrole=cluster-admin --user=$(gcloud config get-value account)
```

取得 Jenkins 叢集的憑證，然後連線至該叢集：

```
gcloud container clusters get-credentials jenkins-cd --zone us-east1-d --project ${GOOGLE_CLOUD_PROJECT}
```

然後將 Helm 二進位檔下載至 Cloud Shell：

```
wget https://storage.googleapis.com/kubernetes-helm/helm-v2.14.1-linux-amd64.tar.gz
```

解壓縮檔案，並將內含的 Helm 檔案複製到目前的工作目錄：

```
tar zxvf helm-v2.14.1-linux-amd64.tar.gz && \
cp linux-amd64/helm .
```

Tiller 是 Helm 的伺服器端，會在 Kubernetes 叢集上執行。讓我們建立名為 `tiller` 的服務帳戶：

```
kubectl create serviceaccount tiller \
--namespace kube-system
```

並將其繫結至 `cluster-admin` 叢集角色，以便進行變更：

```
kubectl create clusterrolebinding tiller-admin-binding \
--clusterrole=cluster-admin \
--serviceaccount=kube-system:tiller
```

現在我們可以初始化 Helm 並更新存放區：

```
./helm init --service-account=tiller && \  
./helm repo update
```



驗證

使用 `./helm version` 確認 Helm 是否正常運作，這應該會傳回用戶端和伺服器的版本號碼：

```
Client: &version.Version{SemVer:"v2.14.1",  
GitCommit:"5270352a09c7e8b6e8c9593002a73535276507c0", GitTreeState:"clean"}  
Server: &version.Version{SemVer:"v2.14.1",  
GitCommit:"5270352a09c7e8b6e8c9593002a73535276507c0", GitTreeState:"clean"}
```

安裝 Jenkins

Helm 安裝在叢集後，我們就可以安裝 Jenkins：

```
./helm install stable/jenkins -n cd \  
-f jenkins/values.yaml \  
--version 1.2.2 --wait
```



驗證

檢查 Pod：

```
kubectl get pods
```

輸出內容應顯示 Jenkins Pod，且狀態為 RUNNING：

NAME	READY	STATUS	RESTARTS	AGE
cd-jenkins-7c786475dd-vbhg4	1/1	Running	0	1m

確認是否已正確建立 Jenkins 服務：

```
kubectl get svc
```

輸出內容應如下所示：

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
cd-jenkins	ClusterIP	10.35.241.170	<none>	8080/TCP	2m27s
cd-jenkins-agent	ClusterIP	10.35.250.57	<none>	50000/TCP	2m27s
kubernetes	ClusterIP	10.35.240.1	<none>	443/TCP	75m

Jenkins 安裝會使用 Kubernetes 外掛程式建立建構工具代理程式。Jenkins 主要執行個體會視需要自動啟動這些代理程式。當這些代理程式的工作完成後，系統就會自動終止代理程式，其中的資源會新增回叢集的資源集區。

連線至 Jenkins

Jenkins 正在叢集上執行，但如要存取 UI，請從 Cloud Shell 設定通訊埠轉送：

```
export POD_NAME=$(kubectl get pods --namespace default -l
"app.kubernetes.io/component=jenkins-master" -l "app.kubernetes.io/instance=cd" -o
jsonpath="{.items[0].metadata.name}") &&
kubectl port-forward $POD_NAME 8080:8080 >> /dev/null &
```

注意：如果關閉目前的 Cloud Shell，就必須重新執行上述連接埠轉送指令。

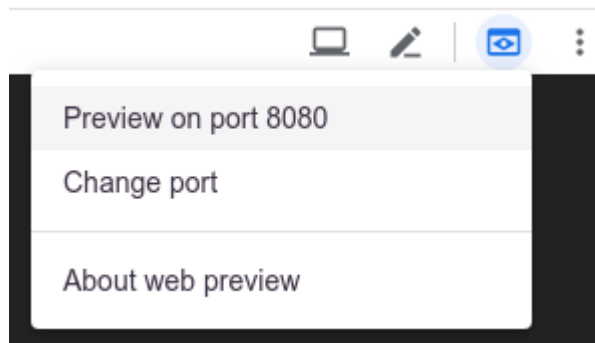
安裝期間會產生管理員密碼。讓我們來擷取：

```
printf $(kubectl get secret cd-jenkins -o jsonpath="{.data.jenkins-admin-password}" |
base64 --decode);echo
```

在 Cloud Shell 頂端，按一下「網頁預覽」圖示



，然後選取「透過以下通訊埠預覽：8080」。



我們應該會看到 Jenkins 的登入畫面，可以在其中輸入使用者名稱 `admin` 和上一個步驟傳回的密碼：



Welcome to Jenkins!

Sign in

按一下「Sign in」後，系統應會將我們導向 Jenkins 的主要頁面。

**Jenkins**

search

admin | log out

Jenkins

[ENABLE AUTO REFRESH](#)

 New Item

 People

 Build History

 Manage Jenkins

 My Views

 Lockable Resources

 Credentials

 New View

Build Queue

No builds in the queue.

[Build Executor Status](#)

 [add description](#)

Welcome to Jenkins!

Please **create new jobs** to get started.

步驟 5 · 約 2 分鐘

5. 安裝及設定 GKE 外掛程式

Google Kubernetes Engine 外掛程式可讓我們將 Jenkins 中建構的部署作業發布至 GKE 執行的 Kubernetes 叢集。您必須在專案中設定 IAM 權限。我們將使用 Terraform 部署該設定。

首先，請下載 GKE 外掛程式專案：

```
git clone https://github.com/jenkinsci/google-kubernetes-engine-plugin.git ~/google-kubernetes-engine-plugin
```

自動設定 IAM 權限

將目前的工作目錄變更為先前複製的 GKE 專案 rbac 目錄：

```
cd ~/google-kubernetes-engine-plugin/docs/rbac/
```

`gcp-sa-setup.tf` 是 Terraform 設定檔，可建立具有受限權限的自訂 GCP IAM 角色，以及要授予該角色的 GCP 服務帳戶。這個檔案需要專案、區域和服務帳戶名稱變數的值。我們首先宣告下列環境變數，提供這些值：

```
export TF_VAR_project=${GOOGLE_CLOUD_PROJECT}
export TF_VAR_region=us-east1-d
export TF_VAR_sa_name=kaniko-role
```

初始化 Terraform、產生計畫並套用：

```
terraform init
terraform plan -out /tmp/tf.plan
terraform apply /tmp/tf.plan && rm /tmp/tf.plan
```

服務帳戶需要儲存空間管理員權限，才能儲存至我們的 Cloud Storage bucket：

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
--member serviceAccount:kaniko-role@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com \
--role 'roles/storage.admin'
```

此外，還需要管道部署階段的容器權限：

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} --member \
serviceAccount:kaniko-role@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com --role \
'roles/container.developer'
```

現在可以使用 Helm，透過 gke 機器人部署者設定 GKE 外掛程式的叢集權限。將工作目錄變更為 GKE 專案的 Helm 目錄：

```
cd ~/google-kubernetes-engine-plugin/docs/helm/
```

然後使用提供的 Helm 資訊套件安裝：

```
export TARGET_NAMESPACE=kube-system && \  
envsubst < gke-robot-deployer/values.yaml | helm install ./gke-robot-deployer --name  
gke-robot-deployer -f -
```

步驟 6 · 約 5 分鐘

6. 設定 Jenkins

服務帳戶金鑰

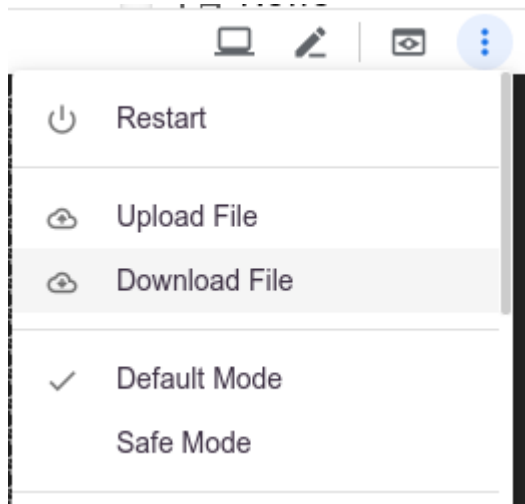
如要讓服務帳戶正常運作，我們需要產生私密金鑰檔案，並將其新增為 Kubernetes 密鑰。首先，請使用下列 gcloud 指令產生檔案：

```
gcloud iam service-accounts keys create /tmp/kaniko-secret.json --iam-account kaniko-role@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com
```

我們會使用該檔案在 Kubernetes Secret 儲存庫中建立 Secret：

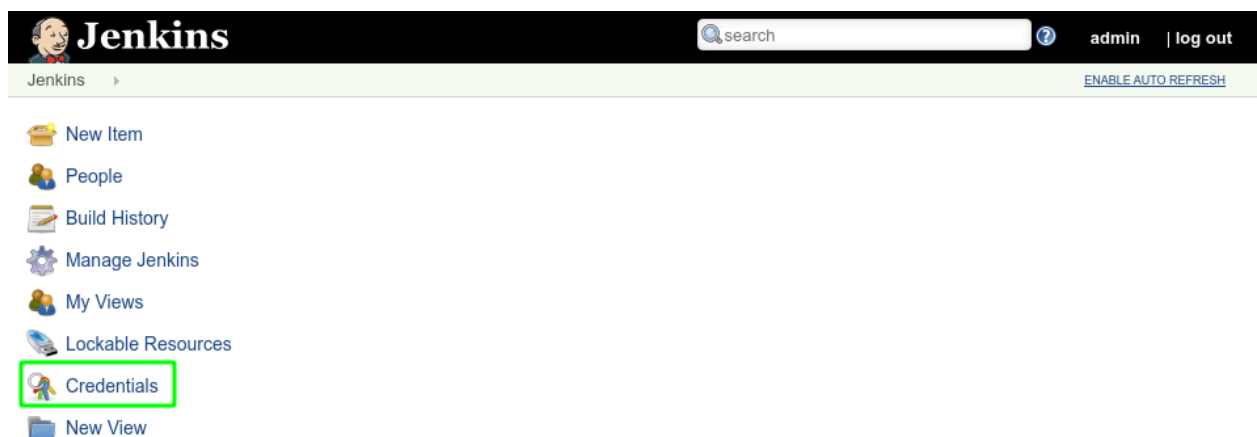
```
kubectl create secret generic jenkins-int-samples-kaniko-secret --from-file=/tmp/kaniko-secret.json
```

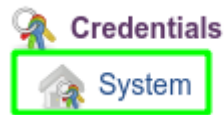
從 Cloud Shell 的 3 點選單存取「下載檔案」項目，將 JSON 檔案下載至本機磁碟：



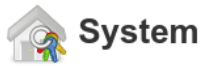
輸入檔案路徑 `/tmp/kaniko-secret.json`，然後按一下「下載」。

返回 Jenkins 頁面，按一下左側窗格的「Credentials」，然後按一下「System」。



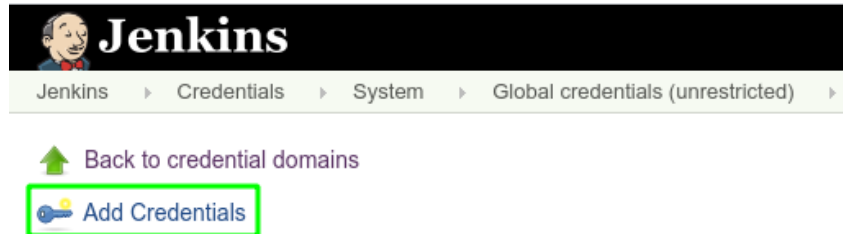


在頁面中標題為「系統」的部分下方，按一下左側的「全域憑證」，然後按一下「新增憑證」：

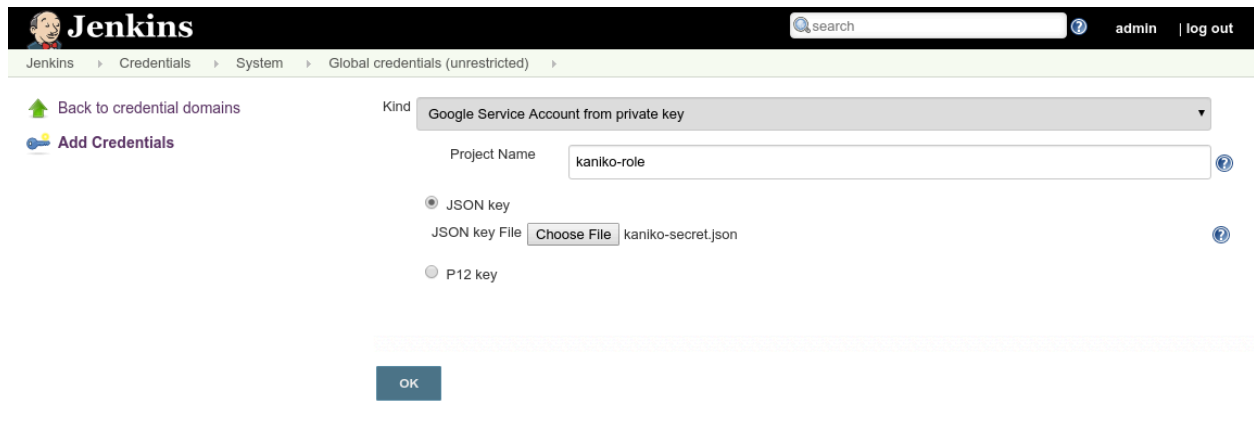


Domain	Description
Global credentials (unrestricted)	Credentials that should be available irrespective of domain specification to requirements matching.

Icon: [S](#) [M](#) [L](#)



在「Kind」下拉式選單中，選取「Google Service Account from private key」。輸入「kaniko-role」做為名稱，然後上傳您在上述步驟中建立的 JSON 金鑰，並按一下「確定」。

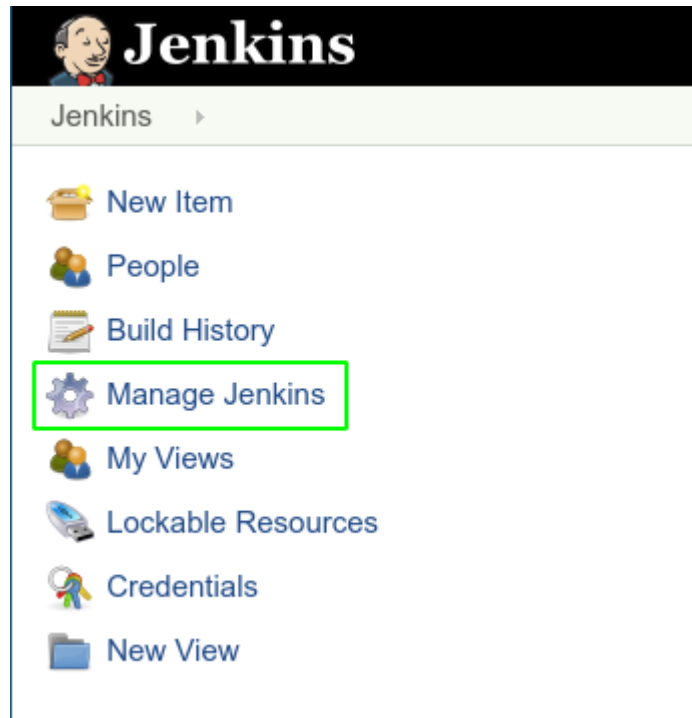


環境變數

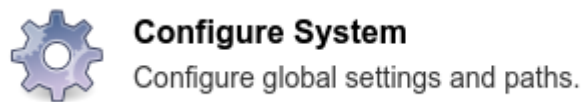
建立多分支 pipeline 前，我們需要先定義 Jenkins 的一些環境變數。這 3 個子類型如下：

- JENK_INT_IT_ZONE - Kubernetes 叢集的區域。在本例中，`us-east1-d`
- JENK_INT_IT_PROJECT_ID - 指的是代管這個 Jenkins 執行個體的 GCP 專案 ID
- JENK_INT_IT_STAGING - 我們的「測試環境」叢集名稱，為示範用途，此名稱為 `staging`
- JENK_INT_IT_PROD - 我們的「prod」叢集名稱。為了進行示範，我們使用 `prod`
- JENK_INT_IT_BUCKET - 在稍早步驟中建立的 Google Cloud Storage bucket
- JENK_INT_IT_CRED_ID - 指的是使用上一個步驟中的 JSON 建立的憑證。這個值應與我們為其指定的名稱相符，即 `kaniko-role`

如要新增這些項目，請前往「Manage Jenkins」：



然後設定系統：



系統會顯示名為「全域屬性」的部分，勾選「環境變數」方塊後，會出現「新增」按鈕，點選即可將上述變數新增為鍵/值組合：

☒ Environment variables

List of variables

Name	JENK_INT_IT_BUCKET	
Value	jenkins-codelab-jenkins-test-bucket	
		Delete
Name	JENK_INT_IT_CRED_ID	
Value	kaniko-role	
		Delete
Name	JENK_INT_IT_PROD	
Value	prod	
		Delete
Name	JENK_INT_IT_PROJECT_ID	?
Value	jenkins-codelab	
		Delete
Name	JENK_INT_IT_STAGING	
Value	staging	
		Delete
Name	JENK_INT_IT_ZONE	
Value	us-east1-d	
		Delete

按一下頁面底部的「儲存」按鈕，即可套用變更。

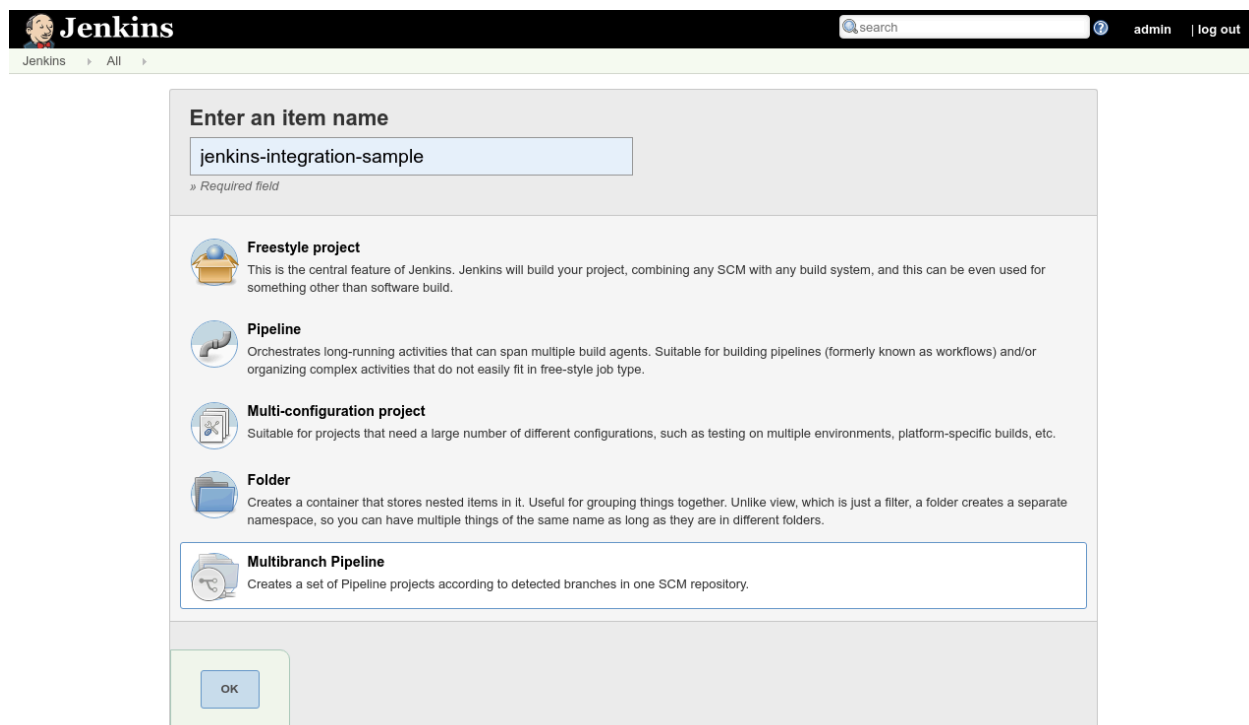
步驟 7 · 約 15 分鐘

7. 設定管道

在 Jenkins 中，按一下「New Item」：



輸入「jenkins-integration-sample」做為名稱，選取「Multibranch Pipeline」做為專案類型，然後按一下「OK」：



系統會將我們重新導向至管道設定頁面。在「Branch Sources」(分支來源) 下方，輸入 <https://github.com/GoogleCloudPlatform/jenkins-integration-samples.git> 做為「Project Repository」(專案存放區)。在「建構設定」下方，輸入「gke/Jenkinsfile」做為「指令碼路徑」。

Branch Sources

Git
✕

Project Repository
?

Credentials - none - ⚙️ Add
?

Behaviors

Discover branches
✕
?

Add

Property strategy All branches get the same properties

Add property

Add source

Mode by Jenkinsfile

Script Path
?

按一下「儲存」即可套用這些設定。儲存後，Jenkins 會啟動存放區掃描，並為每個分支版本進行後續建構。隨著建構作業進行，您會在 Kubernetes 工作負載頁面看到建立、執行及終止的 Pod。

建構完成後，Kubernetes 工作負載頁面會顯示兩個名為 jenkins-integration-samples-gke 的項目，分別對應至正式版或測試版叢集。狀態會顯示「OK」：

Workloads

↻ REFRESH
+ DEPLOY
🗑 DELETE

Workloads are deployable units of computing that can be created and managed in a cluster.

☰
Is system object : False ✕
Filter workloads
✕
?
Columns ▼

☐ Name ^	Status	Type	Pods	Namespace	Cluster
☐ cd-jenkins	✓ OK	Deployment	1/1	default	jenkins-cd
☐ jenkins-integration-samples-gke	✓ OK	Deployment	2/2	default	prod
☐ jenkins-integration-samples-gke	✓ OK	Deployment	2/2	default	staging

使用下列 gcloud 指令，我們會看到已將容器映像檔上傳至對應於管道的 Google Container Registry：

```
gcloud container images list
```

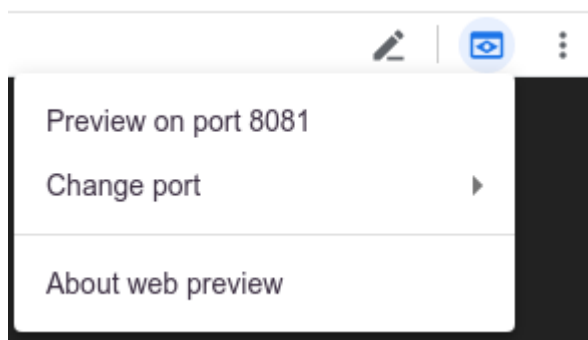
如要在瀏覽器中查看工作負載，請取得生產環境叢集的憑證：

```
gcloud container clusters get-credentials prod --zone us-east1-d --project  
${GOOGLE_CLOUD_PROJECT}
```

然後執行下列指令，將 Shell 的通訊埠 8081 轉送至工作負載的通訊埠 8080：

```
export POD_NAME=$(kubectl get pods -o jsonpath="{.items[0].metadata.name}") &&  
kubectl port-forward $POD_NAME 8081:8080 >> /dev/null &
```

在 Cloud Shell 頂端，按一下「網頁預覽」圖示，然後選取「透過以下通訊埠預覽：8081」



Greetings from Spring Boot!

注意：部分分支機構會暫停，等待輸入內容以核准部署作業。透過工作的狀態頁面核准部署作業。

步驟 8 · 約 4 分鐘

8. 清除所用資源

我們已探討如何在 Kubernetes 上部署 Jenkins 和範例多分支管道。現在要清理專案中建立的所有資源。

刪除專案

如有需要，也可以刪除整個專案。前往 GCP 主控台的「Cloud Resource Manager」頁面：

在專案清單中，選取我們一直在處理的專案，然後按一下「刪除」。系統會提示您輸入專案 ID。輸入後，按一下「關機」。

或者，您也可以直接透過 Cloud Shell 和 gcloud 刪除整個專案：

```
gcloud projects delete ${GOOGLE_CLOUD_PROJECT}
```

如要逐一刪除不同的計費元件，請參閱下一節。

Kubernetes 叢集

使用 gcloud 刪除整個 Kubernetes 叢集：

```
gcloud container clusters delete jenkins-cd --zone=us-east1-d
```

儲存空間 Bucket

移除所有上傳的檔案，並使用 gsutil 刪除我們的 bucket：

```
gsutil rm -r gs://${GOOGLE_CLOUD_PROJECT}-jenkins-test-bucket
```

Google Container Registry 映像檔

我們會使用映像檔摘要刪除 Google Container Registry 映像檔。首先，使用下列指令擷取摘要：

```
gcloud container images list-tags gcr.io/${GOOGLE_CLOUD_PROJECT}/jenkins-integration-samples-gke --format="value(digest)"
```

然後針對傳回的每個摘要執行下列操作：

```
gcloud container images delete gcr.io/${GOOGLE_CLOUD_PROJECT}/jenkins-integration-samples-gke@sha256:<DIGEST>
```

步驟 9 · 約 0 分鐘

9. 恭喜！

太厲害了！你做到了。您已瞭解如何在 GKE 上部署 Jenkins，並將工作指派給 Kubernetes 叢集。

涵蓋內容

- 我們已部署 Kubernetes 叢集，並使用 Helm 安裝 Jenkins
 - 我們已安裝及設定 GKE 外掛程式，讓 Jenkins 能將建構構件部署到 Kubernetes 叢集
 - 我們已設定 Jenkins，建立可將工作分派至 GKE 叢集的多分支管道
-